

Развёртывание и интеграция с Битриксом

Документ для IT-отдела. Что положить на сервер, как передать данные заказа, какие заголовки выставить и что ещё предстоит дописать на бэкенде.

Описание самого проекта и его настроек — в [../README.md](#).

1. Что разворачивать

Node на сервере **не нужен**: код уже собран. Копируются файлы:

<code>index.html</code>	
<code>dist/</code>	собранный JS
<code>css/</code>	
<code>assets/</code>	3D-модели, макеты, карта освещения, иконки
<code>vendor/</code>	распаковщик Draco

Всё остальное (`js/` , `scripts/` , `node_modules/` , `docs/`) — исходники и на сервере не нужны.

Страница статическая: серверного кода в ней нет, она только читает данные и показывает картинку.

Куда именно её класть

Возможны два варианта, оба рабочие:

(а) Отдельный каталог на том же домене — например `/approve/`. Битрикс только генерирует ссылку, страница отдаётся веб-сервером как статика.

(б) Шаблон/компонент Битрикса — разметка из `index.html` переносится в шаблон, PHP печатает в неё данные заказа.

Важно про пути. Сейчас все пути относительные (`assets/...` , `dist/...`). Они работают, если страница лежит по адресу, заканчивающемуся слэшем (`/approve/`), и ломаются при `/approve` без слэша или при выносе статики на отдельный домен. Если выбран вариант (б) или адрес другой — скажите, добавим настраиваемый базовый путь.

2. Как передать данные заказа

Способ 1 — напечатать объект в страницу (рекомендуется)

Шаблон выводит объект **до** подключения `dist/cup-viewer.js`. Ни одного лишнего запроса: данные уже в HTML персональной ссылки.

```
<script nonce="<?= $nonce ?>">
window.ORDER_DATA = <?= json_encode($order,
    JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT | JSON_UNESCAPED_UNICODE) ?>;
</script>
<script type="module" src="dist/cup-viewer.js"></script>
```

Объект обязан собираться через `json_encode`, а не конкатенацией строк. В полях заказа встречаются название клиента, комментарий менеджера и имя файла: кавычка или последовательность `</script>` внутри них даст исполнение чужого кода ровно на той странице, где клиент согласовывает макет. Флаг `JSON_HEX_TAG` закрывает именно этот случай.

Форма объекта:

```
window.ORDER_DATA = {
  sku: 'DW80-280',
  paper: 'uncoated',
  texture: '/upload/orders/2431/design.png',
  unwrap: '/upload/orders/2431/design-print.png',
  status: 'Ожидает вашего согласования',
  title: 'Посмотрите свой стакан вживую',
  spec: [
    { label: 'Тип', value: 'Двухслойный' },
    { label: 'Объём', value: '250 мл (до 280 мл)' },
    { label: 'Печать', value: 'Офсет' },
    { label: 'Тираж', value: '5 000 шт.' }
  ],
  checklist: [
    'Логотип и его пропорции соответствуют брендбуку',
    'Цвета в макете: Pantone 485 C и 100% чёрный'
  ]
};
```

Способ 2 — запрос за данными

Ссылка вида `/approve/?order=2431`. В `js/config.js` указывается адрес, откуда забрать JSON (`{id}` подставляется из адреса):

```
order: {
  endpoint: '/local/api/viewer-order.php?id={id}',
},
```

Ответ — тот же объект. Требования: `Content-Type: application/json` , ответ не кешируется. Если запрос не удался, страница покажет значения по умолчанию, а не пустоту.

Способ 3 — параметры адреса (только отладка)

```
?sku=DW80-280&paper=uncoated&texture=/upload/design.png
```

Работают **только на localhost** . На боевом домене отключены намеренно: иначе по ссылке `?texture=...&title=Заказ+аннулирован` клиенту можно показать чужой макет и любой текст на вашем же домене, без единого признака подделки.

3. Поля заказа

Все необязательны — чего нет, берётся из `DEFAULT_ORDER` в `js/data/order.js` .

Поле	Тип	Что делает
<code>sku</code>	строка	Номенклатура. По ней подбирается 3D-модель (<code>DW80-280</code> и т.п.)
<code>model</code>	строка	Прямой путь к <code>.glb</code> . Задан — важнее <code>sku</code>
<code>texture</code>	строка	Путь к макету. Ложится на модель и открывается во вкладке «Развёртка»
<code>unwrap</code>	строка	Отдельный крупный файл для вкладки «Развёртка». Не задан — берётся <code>texture</code>
<code>paper</code>	строка	<code>coated</code> / <code>uncoated</code> (принимает и «мелованный» / «немелованный»)
<code>roughness</code>	число	Шероховатость 0...1 вручную. Задана — важнее <code>paper</code>
<code>status</code>	строка	Текст плашки состояния
<code>title</code>	строка	Заголовок панели
<code>lead</code>	строка	Абзац под заголовком
<code>spec</code>	массив	Строки таблицы: <code>[{ label, value }, ...]</code>
<code>checklist</code>	массив	Пункты чек-листа перед печатью: массив строк
<code>note</code>	строка	Мелкий текст-оговорка под кнопками

Всё, что пришло снаружи, проверяется в `js/data/sanitize.js` : незнакомые поля отбрасываются, строки обрезаются по длине, пути к файлам принимаются только с разрешённых префиксов (`config.security.allowedAssetPrefixes` , по умолчанию `assets/` и `/upload/`). Отклонённое поле заменяется значением по умолчанию, а причина пишется в консоль браузера.

Про размер макета

Просмотрщик уменьшает файл, который кладётся на модель, до 2048 px по длинной стороне: печатное разрешение (7000 px — норма) видеокарта телефона либо не примет, либо займёт под него почти всю память.

Крупный исходник для проверки мелкого текста лучше отдавать отдельным полем — он показывается плоско и на видеокарту не попадает:

```
texture: '/upload/orders/2431/design-1024.jpg', // на модель
unwrap: '/upload/orders/2431/design-print.png', // во вкладку «Развёртка»
```

Номенклатура и картон

Правятся в `js/data/catalog.js` :

```
export const MODELS = {
  'DW80-280': 'assets/models/8cups.glb',
  'DW90-430': 'assets/models/8cups.glb',
};

export const PAPER_TYPES = {
  coated: { label: 'Мелованный', roughness: 0.2 },
  uncoated: { label: 'Немелованный', roughness: 0.8 },
};
```

Если точного ключа нет, номенклатура разбирается по префиксу (`DW...` , `HB...`), а если и это не сработало — грузится модель по умолчанию и в консоль уходит предупреждение. Что именно сработало, видно в `cupViewer.order.modelMatchedBy` : `sku` , `pattern` , `explicit` или `fallback` .

Сейчас все номенклатуры ведут на одну модель-заглушку `8cups.glb` . Реальные модели и полный список позиций нужно завести отдельно.

4. Заголовки ответа

```

add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "same-origin" always;
add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

```

Тот же набор ставит локальный сервер разработки (`scripts/serve.mjs`), чтобы расхождение конфигураций обнаруживалось на своей машине, а не у клиента.

На что обратить внимание:

Директива	Почему именно так
<code>connect-src 'self' blob:</code>	blob: обязателен. Загрузчик 3D-моделей читает встроенные в <code>.glb</code> текстуры по <code>blob</code> -адресу, а <code>'self'</code> на схему <code>blob:</code> не распространяется. Без него текстура молча не грузится и макет не ложится на стакан вообще — выглядит как «просмотрщик показывает белый стакан», ищется долго
<code>worker-src 'self' blob:</code>	Распаковщик Draco работает в фоновом потоке, создаваемом из <code>blob</code>
<code>frame-ancestors 'none'</code>	Без него кнопку «Согласовать» можно подложить под прозрачный <code>iframe</code> на чужом сайте
<code>script-src 'self'</code>	Внешних скриптов на странице нет. При способе 1 (объект печатается в страницу) понадобится <code>'nonce-...'</code> — <code>nonce</code> генерируется заново на каждый ответ

Кеширование

Персональная ссылка не должна попадать ни в один общий кеш, включая композитный кеш Битрикса:

```
Cache-Control: no-store, private
```

Ассеты (`assets/` , `dist/` , `vendor/`), наоборот, кешируются надолго — сборка дописывает к их адресам метку версии, поэтому обновление доезжает само:

```
Cache-Control: public, max-age=31536000, immutable
```

Если макеты лежат на другом домене

Тогда на хранилище обязан быть настроен CORS:

Access-Control-Allow-Origin: https://<домен-страницы>

Без этого браузер откажется класть изображение на 3D-модель, и клиент увидит белый стакан без объяснения причин. Заодно проверьте `Content-Type` у файлов макетов: с `nosniff` картинка, отданная как `application/octet-stream`, не отобразится.

5. Что осталось сделать на бэкенде

Две вещи, которые невозможно закрыть на стороне страницы.

5.1. Защита ссылки

Сейчас в примерах адрес вида `/approve/?order=2431`. Номера заказов последовательны: получив одну ссылку, человек подставит соседние номера и увидит чужие макеты — логотипы до релиза, контакты, состав, тираж.

Что нужно:

1. **Непредсказуемый идентификатор** — случайный токен не менее 128 бит (`random_bytes(16)` и выше) либо HMAC от номера заказа на серверном секрете. Номер заказа в адресе — только вместе с токеном, никогда вместо него.
2. **Серверная проверка** при каждой отдаче данных: токен → заказ, а не наоборот.
3. **Срок жизни** и возможность отозвать ссылку.
4. **Ограничение частоты запросов** — от перебора.
5. Если кнопка «Поделиться ссылкой» остаётся — она должна отдавать ссылку **только на просмотр**, иначе клиент своими руками раздаёт право согласовать макет от его имени.

5.2. Собственно согласование

Сейчас нажатие «Согласовать и запустить в печать» меняет надпись и показывает сообщение. **На сервер не уходит ничего.** То же с формой правок: текст берётся и выбрасывается. При этом на странице написано «Согласование фиксирует макет: после запуска в печать правки невозможны».

Что нужно:

Требование

`POST` с токеном доступа

Зачем

Собственно фиксация факта

Требование	Зачем
Защита от CSRF	Иначе согласование можно вызвать со стороннего сайта
Идемпотентность (ключ запроса)	Двойной клик и кнопка «назад» не должны создавать два согласования
Запись: время, IP, User-Agent, версия макета, состояние каждой галочки	Доказательство в спорной ситуации
Хеш согласованного файла в записи	Чтобы файл нельзя было подменить задним числом
Письмо-подтверждение клиенту	Он должен получить копию того, что согласовал
Отдельный приёмник для правок	Текст правок должен доходить до менеджера

Пока этого нет, интерфейс вводит клиента в заблуждение. Если запуск планируется раньше бэкенда — кнопку согласования лучше заблокировать с честной подписью, чем показывать успех.

6. Если не работает

Симптом	Причина
Белый стакан без макета	В CSP нет <code>blob:</code> в <code>connect-src</code> — см. раздел 4
Макет не появился, в консоли «поле "texture" отклонено»	Путь не проходит по <code>security.allowedAssetPrefixes</code> или ведёт на другой домен
Макет не появился, ошибка CORS	Хранилище макетов на другом домене без <code>Access-Control-Allow-Origin</code>
Данные заказа не применились	Откройте консоль: <code>cupViewer.order</code> покажет, что дошло; отклонённые поля пишутся туда же
Пустая страница на старом iPhone	Не подключён собранный <code>dist/cup-viewer.js</code> , а исходники из <code>js/</code>
Модель по умолчанию вместо нужной	Номенклатура не опознана, см. <code>cupViewer.order.modelMatchedBy</code>
Изменения в коде не видны	Не выполнен <code>npm run build</code> — браузер читает <code>dist/</code>